

Parallel and Distributed Systems

Chapter 7: Asynchronous Programming

Prof. Dr. Martin Sträßer



Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted

Recap: Serial Execution

- Until 1980s, computers ran one program at a time, instruction after instruction, in a strictly sequential manner
- This is called *serial* or *sequential execution*

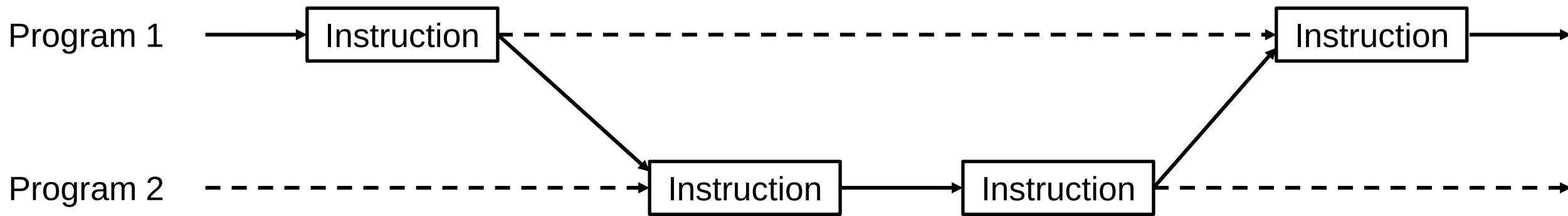


- (Only) key performance indicator for CPUs was frequency in Hz (instructions per second)
- Intel 80386 processor was one of first processors that allowed to interrupt a program in order to work on another
- This allows *time sharing* between different programs



Recap: Concurrent Execution

- Switching between and resuming programs allows us manage multiple programs *at the same time, but not simultaneously*
- This is called *concurrent execution*

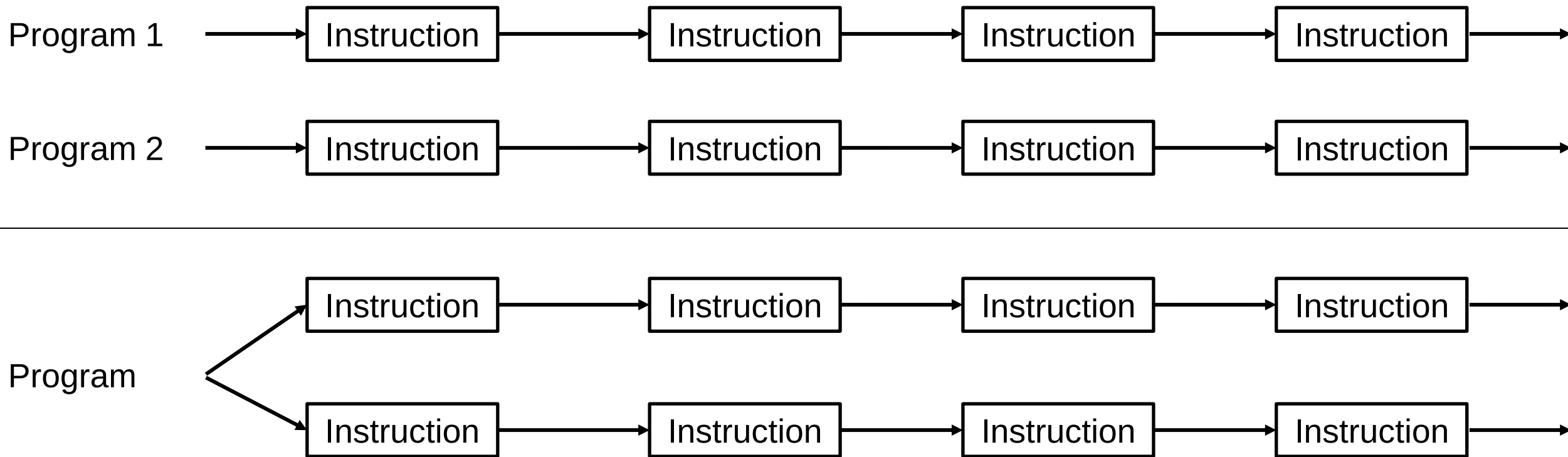


- If the instruction processing and process switching takes place at a high frequency, it *seems* that multiple programs are executed *in parallel*



Recap: Parallel Execution

- Given that we have a multi-core system, we can process multiple instructions from a single or multiple programs at the same time
- This is called *parallel* or *true parallel execution*





Overview

- Important Concepts in Asynchronous Programming
- Asynchronous Programming in Python

Important Concepts in Asynchronous Programming



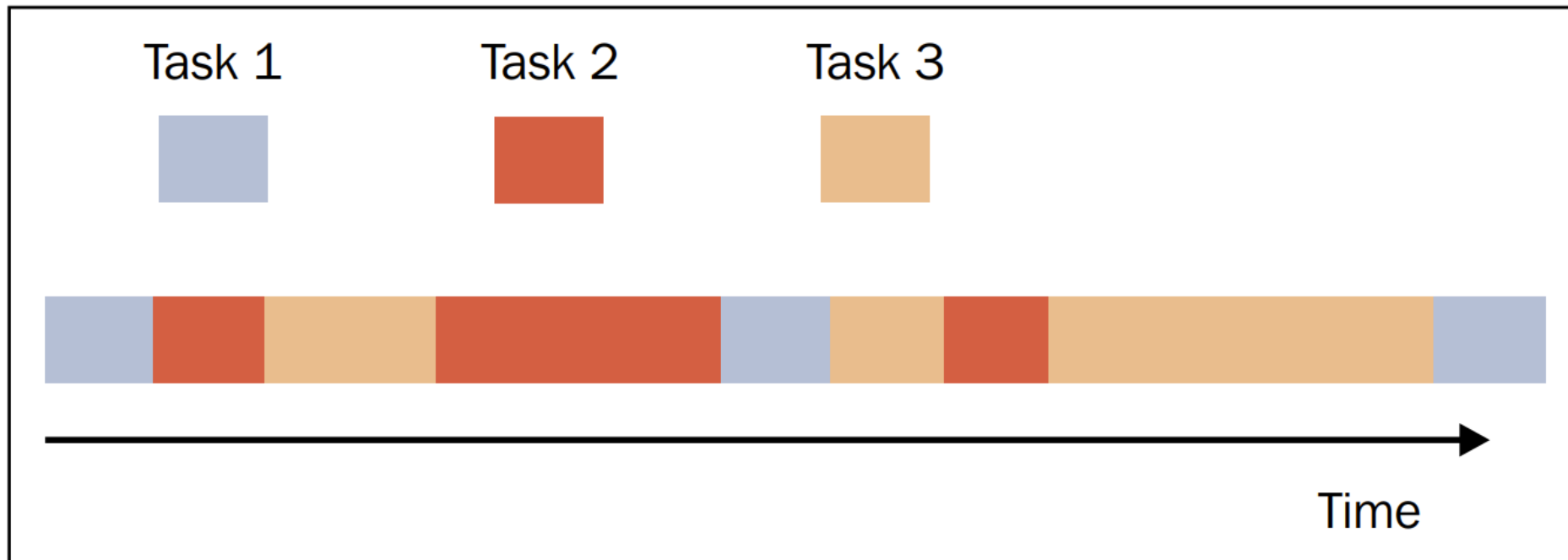
Asynchronous Programming

- Asynchronous programming is a style of concurrent programming and an alternative to threads and processes
- Asynchronous programming is **single-threaded concurrency**
- Consequences
 - (Fully) asynchronous programs run on a single core at a time
 - (Fully) asynchronous programs cannot use parallel execution
 - There are no race conditions, no synchronization
- Also called **cooperative multitasking**



Asynchronous Programming

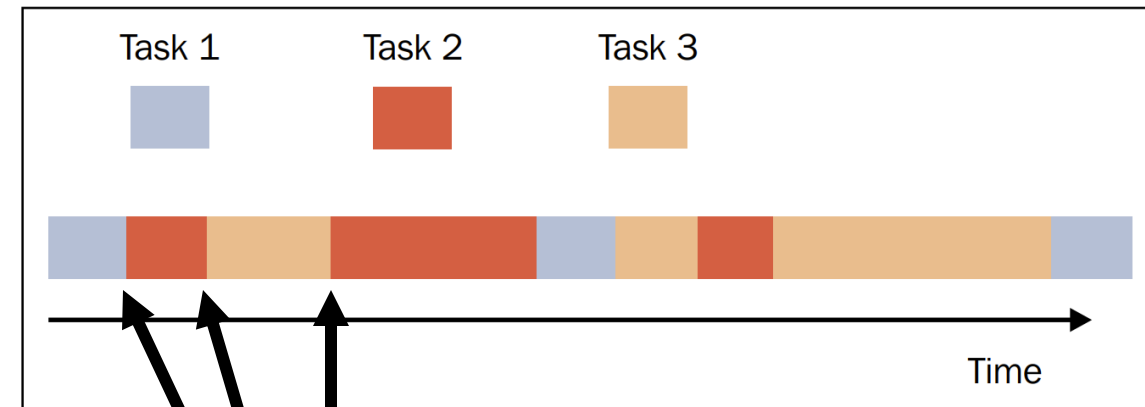
- In asynchronous programming, various tasks intersect with the timeline, and all of this happens under the action of a single flow of control (single-threaded)
- The execution of a task can be suspended and then resumed, but this alternates the time of other tasks





Asynchronous Programming

- The tasks (each with a different color) are interleaved with one another, but they are in a single thread of control
- This implies that when one task is in execution, the other tasks are not
- In multi-threaded or multi-process programs, the operating system decides on the timeline
- In asynchronous programming, the tasks voluntarily yield the execution

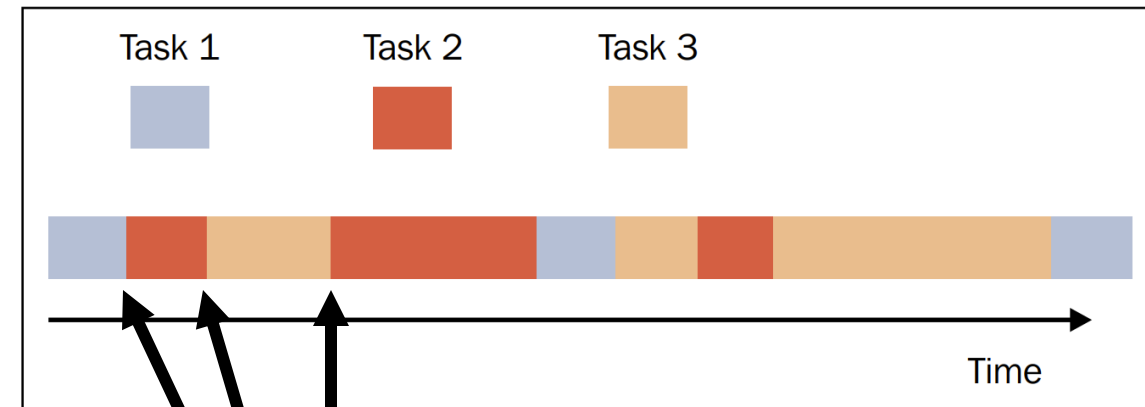


Switches between tasks are caused by the tasks themselves. They give the signal that another task may run now.



Asynchronous Programming

- Questions: How can a task yield its execution? Who decides which task runs next, if it is not the operating system?
- Solution: The event loop



Switches between tasks are caused by the tasks themselves. They give the signal that another task may run now.

Event Loop

- Events are a form of asynchronous communication
- We distinguish event sources and event handlers, both do not know about the existence of each other
- Event loop manages all the events and calls event handlers





Event Loop

- Pseudocode of an event loop

```
while True:
    events = getEvents();
    for e in events:
        processEvent(e);
```

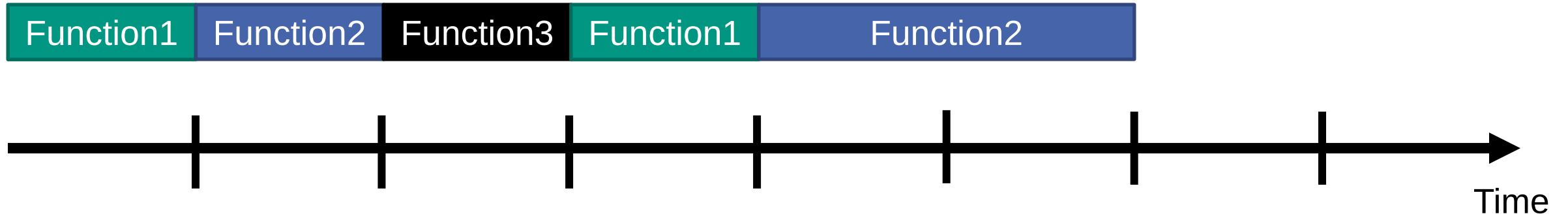
- All the events in the while loop are caught and then processed sequentially by event handlers
- When the handler has ended, the control is passed on to the next event that is scheduled
- Event handlers or external entities may create new events



Example

- **Function1:**
 1. Computes something for 1 second
 2. Needs to wait for 2 seconds, creates a “ResumeAfter2SecondsEvent”
 3. Yields its execution
 4. Computes something for 1 second
- **Function2:**
 1. Computes something for 1 second
 2. Needs to wait for 1 seconds, creates a “ResumeAfter1SecondEvent”
 3. Yields its execution
 4. Computes something for 2 seconds
- **Function3**
 - Computes something for 1 second

Example



- Things to see
 - Concurrent execution with a single flow of control (one thread)
 - No timing guarantees (e.g., Function2 could have resumed after 1 second break, but is resumed after 2 seconds)



Coroutines

- Functions used for asynchronous programming are called **coroutines**
- A coroutine signals by itself if it is waiting for an external event
- It is cooperative and yields the control flow to another function
- Coroutines are processed sequentially (one instruction after another)
- Having a pool of coroutines, it is possible to interleave their computations: run the first one until it yields the control back, then run the second, and so on (=asynchronous programming)



Tasks

- What if a coroutine waits for results from another coroutine?
- A task is a scheduling wrapper around a coroutine that tells the event loop to run it
- A coroutine is just an object describing what could run
- A task:
 - Starts the execution of a coroutine
 - Manages resuming after execution is yielded
 - Handles completion, cancellation, and failures in a coroutine
- Multiple tasks:
 - Are conceptually independent
 - Compete for execution time



Recap: Futures/Promises

- Option 3: Futures/Promises
 - The return value of the non-blocking IPC operation is a future, a specific data type
 - A future is like a box that we can treat as a normal variable
 - Once we need the response, we „open the box“
 - If the response is there, we extract it and continue, if not, we wait until the response is there
- Analogy: We get an order number from the waiter. When we ended our conversation with our friends, we go to the pick-up station. We may wait there, if our food is not ready.



Futures

- Futures are the return value of coroutines/tasks
- Tasks, coroutines, and futures are so-called **awaitable objects** (i.e., their current state is unknown and another entity may wait its completion)
- When we invoke an asynchronous operation, a future is returned immediately as the return type
- When we need to know the value of that future, we **await** that future



Futures

- Behavior when we await a future
 - If the future is already completed, the future returns its value immediately
 - If the future has failed/threw an exception, that exception is thrown immediately
 - If the future is not yet completed, the execution is yielded and resumed if the future is completed
- Multiple coroutines/tasks can wait for the same future



Question

- What are advantages and disadvantages of asynchronous programming?



Pros and Cons

- Pros

- More responsive to external events
- If well designed, optimizes the control flow doable with a single thread
- Keeps many benefits of sequential execution (no race conditions, synchronization etc.)
- Good for inherently event-driven programs, such as GUIs and web servers

- Cons

- No predetermined execution
- Bad if we often need results from other coroutines immediately
- Complex implementation, code comprehension, and maintenance



Async Programming in Different Languages

- JavaScript/TypeScript
 - Single-threaded, fully based on event loop
 - Understanding async programming is essential
- Python
 - Popular as it circumvents the GIL limitation of threads
- C++/C#/Java
 - Optional to use, but upcoming trend
- Go
 - No support, goroutines are superlight threads

Async Programming in Python

- Popular Python library: asyncio (part of standard library)
- Core Components
 - Event loop
 - Coroutines
 - Futures
 - Tasks
- Relatively new, introduced in Python 3.7
- Regular major (breaking) changes to library (e.g., in Python 3.8 and 3.10)
- We will see both old asyncio code (for better understanding) and up-to-date asyncio code

Manual Interaction with the Event Loop

- We start with the most important component: the event loop
- Important methods:
 - `get_event_loop()`: Get the event loop for the current context
 - `loop.call_later(time_delay, callback, argument)`: This arranges for the callback that is to be called after the given `time_delay` seconds
 - `loop.call_soon(callback, argument)`: This arranges for a callback that is to be called as soon as possible.
 - `loop.time()`: This returns the current time, as a float value, according to the event loop's internal clock
 - `asyncio.set_event_loop()`: Sets the event loop for the current context
 - `asyncio.new_event_loop()`: Returns a new event loop object
 - `loop.run_forever()`: This runs until `stop()` is called.

Manual Interaction with the Event Loop

OLD

```
import asyncio
```

```
def function_1(end_time, loop): 2 usages
    print("function_1 called")
    if (loop.time() + 1.0) < end_time:
        loop.call_later(1, function_2, end_time, loop)
    else:
        loop.stop()
```

```
def function_2(end_time, loop): 1 usage
    print("function_2 called")
    if (loop.time() + 1.0) < end_time:
        loop.call_later(1, function_3, end_time, loop)
    else:
        loop.stop()
```

```
def function_3(end_time, loop): 1 usage
    print("function_3 called")
    if (loop.time() + 1.0) < end_time:
        loop.call_later(1, function_1, end_time, loop)
    else:
        loop.stop()
```

```
def function_3(end_time, loop): 1 usage
    print("function_3 called")
    if (loop.time() + 1.0) < end_time:
        loop.call_later(1, function_1, end_time, loop)
    else:
        loop.stop()
```

```
if __name__ == '__main__':
    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)

    end_loop = loop.time() + 9.0
    loop.call_soon(function_1, *args: end_loop, loop)

    try:
        loop.run_forever()
    finally:
        loop.close()
```

See code no. 1



Cooperative Tasks (Python <3.8)

OLD

```
import asyncio

@asyncio.coroutine
def factorial(number):
    f = 1
    for i in range(2, number + 1):
        print(f"Asyncio.Task: Compute factorial({i})")
        yield from asyncio.sleep(1)
        f *= i
    print(f"Asyncio.Task - factorial({number}) = {f}")

@asyncio.coroutine
def fibonacci(number):
    a, b = 0, 1
    for i in range(number):
        print(f"Asyncio.Task: Compute fibonacci ({i})")
        yield from asyncio.sleep(1)
        a, b = b, a + b
    print(f"Asyncio.Task - fibonacci({number}) = {a}")
```

```
@asyncio.coroutine
def binomialCoeff(n, k):
    result = 1
    for i in range(1, k + 1):
        result = result * (n - i + 1) / i
    print(f"Asyncio.Task: Compute binomialCoeff ({i})")
    yield from asyncio.sleep(1)
    print(f"Asyncio.Task - binomialCoeff({n} , {k}) = {result}")

if __name__ == "__main__":
    tasks = [asyncio.Task(factorial(10)),
             asyncio.Task(fibonacci(10)),
             asyncio.Task(binomialCoeff( n: 20, k: 10))]
    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)
    loop.run_until_complete(asyncio.wait(tasks))
    loop.close()
```

See code no. 2

Cooperative Tasks (Python 3.8+)

NEW

```
import asyncio
```

```
async def factorial(number): 1 usage
    f = 1
    for i in range(2, number + 1):
        print(f"Asyncio.Task: Compute factorial({i})")
        f *= i
        await asyncio.sleep(1)
    print(f"Asyncio.Task - factorial({number}) = {f}")
```

```
async def fibonacci(number): 1 usage
    a, b = 0, 1
    for i in range(number):
        print(f"Asyncio.Task: Compute fibonacci({i})")
        a, b = b, a + b
        await asyncio.sleep(1)
    print(f"Asyncio.Task - fibonacci({number}) = {a}")
```

```
async def binomialCoeff(n, k): 1 usage
    result = 1
    for i in range(1, k + 1):
        result = result * (n - i + 1) / i
        print(f"Asyncio.Task: Compute binomialCoeff({i})")
        await asyncio.sleep(1)
    print(f"Asyncio.Task - binomialCoeff({n}, {k}) = {result}")
```

```
async def main(): 1 usage
    tasks = [
        asyncio.create_task(factorial(10)),
        asyncio.create_task(fibonacci(10)),
        asyncio.create_task(binomialCoeff(n=20, k=10)),
    ]
    await asyncio.gather(*tasks)
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

See code no. 3

Tasks vs. Coroutines

NEW

```
import asyncio
import time

async def other(id, t): 3 usages
    await asyncio.sleep(t)
    print(f"I am coroutine {id}")

async def main(): 1 usage
    t1 = time.perf_counter()
    await other(id: 1, t: 10)
    await other(id: 2, t: 4)
    await other(id: 3, t: 1)
    t2 = time.perf_counter()
    elapsed_time = t2 - t1
    print(f"Elapsed time {elapsed_time}")

if __name__ == '__main__':
    asyncio.run(main())
```

See code no. 4a

Tasks vs. Coroutines

NEW

```
import asyncio
import time

async def other(id, t): 3 usages
    await asyncio.sleep(t)
    print(f"I am coroutine {id}")

async def main(): 1 usage
    t1 = time.perf_counter()
    await asyncio.gather(other(id: 1, t: 10), other(id: 2, t: 4), other(id: 3, t: 1))
    t2 = time.perf_counter()
    elapsed_time = t2 - t1
    print(f"Elapsed time {elapsed_time}")

if __name__ == '__main__':
    asyncio.run(main())
```

See code no. 4b



Coroutines

- Coroutines in Python are objects
- If you try to make a call to the coroutine by itself, like any other function: `coroutine()`, you will see that you get an warning and that has never been performed
- Coroutines must always been awaited directly or indirectly
- Execution options for coroutines:
 - `asyncio.run()`
 - Called from other coroutine
 - `create_task`
- Tasks allow the await coroutines



asyncio Futures

- Creating a future manually: `future = asyncio.Future()`
- The basic methods of this class are:
 - `cancel()`: This cancels the future and schedules callbacks
 - `result()`: This returns the result that this future represents
 - `exception()`: This returns the exception that was set on this future
 - `add_done_callback(fn)`: This adds a callback that is to be run when future is executed
 - `set_result(result)`: This marks the future as complete and sets its result
 - `set_exception(exception)`: This marks the future as complete and sets an exception

asynio Futures (Python <3.8)

OLD

```
import asynio
```

```
@asynio.coroutine 1 usage
```

```
def sum_of_n(future, N):
    count = 0
    for i in range(1, N + 1):
        count = count + i
    yield from asynio.sleep(4)
    future.set_result("first coroutine result = " \
                      + str(count))
```

```
@asynio.coroutine 1 usage
```

```
def factorial(future, N):
    count = 1
    for i in range(2, N + 1):
        count *= i
    yield from asynio.sleep(3)
    future.set_result("second coroutine result = " \
                      + str(count))
```

```
def got_result(future): 2 usages
    print(future.result())
```

```
if __name__ == "__main__":
    N1 = 10
    N2 = 10
    loop = asynio.new_event_loop()
    asynio.set_event_loop(loop)
    future1 = asynio.Future()
    future2 = asynio.Future()
    tasks = [
        sum_of_n(future1, N1),
        factorial(future2, N2)]
    future1.add_done_callback(got_result)
    future2.add_done_callback(got_result)
    loop.run_until_complete(asynio.wait(tasks))
    loop.close()
```

See code no. 5

asynio Explicit Futures (Python 3.8+)

OLD

```
import asynio
```

```
async def sum_n(future, N): 1 usage
    count = 0
    for i in range(1, N + 1):
        count += i
    await asynio.sleep(4)
    future.set_result("first coroutine result = " + str(count))
```

```
async def factorial(future, N): 1 usage
    count = 1
    for i in range(2, N + 1):
        count *= i
    await asynio.sleep(3)
    future.set_result("second coroutine result = " + str(count))
```

```
def got_result(future): 2 usages
    print(future.result())
```

```
async def main(): 1 usage
    N1 = 10
    N2 = 10
    future1 = asynio.get_running_loop().create_future()
    future2 = asynio.get_running_loop().create_future()
    future1.add_done_callback(got_result)
    future2.add_done_callback(got_result)
    tasks = [
        asynio.create_task(sum_n(future1, N1)),
        asynio.create_task(factorial(future2, N2)),
    ]
    await asynio.gather(*tasks)
```

```
if __name__ == "__main__":
    asynio.run(main())
```

See code no. 6

asyncio Implicit Futures (Python 3.8+)

NEW

```
import asyncio
```

```
async def sum_n(N): 1 usage
    count = sum(range(1, N + 1))
    await asyncio.sleep(4)
    return f"first coroutine (sum of N integers) result = {count}"
```

```
async def factorial(N): 1 usage
    count = 1
    for i in range(2, N + 1):
        count *= i

    await asyncio.sleep(3)
    return f"second coroutine (factorial) result = {count}"
```

```
async def main(): 1 usage
    results = await asyncio.gather(sum_n(10), factorial(10))
    for result in results:
        print(result)
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

See code no. 7



Concurrent Futures != asyncio Futures

- In Python, both `asyncio.futures` and `concurrent.futures` exist
- Despite the conceptual similarity, one must be very careful not to confuse these two types of Future classes
- The futures of the `concurrent` module are based on a multi-threaded construct, while those of `asyncio` are strongly single-threaded
- So, although semantically they perform the same tasks, they work completely differently and cannot be interchanged with each other



Asynchronous Iteration

```
import asyncio

async def f(i): 1 usage
    print(f"start iteration step {i}")
    await asyncio.sleep(1)
    print(f"end iteration step {i}")
    return i

async def main(): 1 usage
    for j in asyncio.as_completed([f(i) for i in range(10)]):
        result = await j
        print(f"result is {result}")

if __name__ == '__main__':
    asyncio.run(main())
```

See code no. 8



Asynchronous Queue

```
import asyncio
import random

async def producer(name, queue): 1 usage
    n = random.randint(a: 0, b: 10)
    await asyncio.sleep(n)
    await queue.put(n)
    print(f"Producer {name} adds {n} to the queue")

async def consumer(name, queue): 1 usage
    while True:
        n = await queue.get()
        await asyncio.sleep(n)
        print(f"Consumer {name} receives {n} from the queue")
        queue.task_done()
```

```
async def main(nproducers, nconsumers): 1 usage
    q = asyncio.Queue()
    producers = [asyncio.create_task(producer(n, q)) for n in
                  range(nproducers)]
    consumers = [asyncio.create_task(consumer(n, q)) for n in
                 range(nconsumers)]

    await asyncio.gather(*producers)
    await q.join()
    for c in consumers:
        c.cancel()

if __name__ == '__main__':
    asyncio.run(main(nproducers: 4, nconsumers: 2))
```

See code no. 9



Blocking Operations in Coroutines

- Until now, we were able to fully write our program code asynchronously
- No blocking (long running, not interruptable) operation happened
- Sometimes blocking operations are unavoidable:
 - CPU-bound calculations
 - I/O operation that cannot be async executed (there are async libraries for HTTP and files, but they are not conceptually async, they use multiple threads)
 - We are executing other non-async code
- Problem: Blocking operations block the whole event loop
- Solution: Spawn new thread/process for the blocking task



Blocking Operations in Coroutines

```
import asyncio
import time

def process(i): 1 usage
    print(f"Processing {i}")
    time.sleep(2)

async def f(i): 1 usage
    print(f"start iteration step {i}")
    #process(i)
    loop = asyncio.get_event_loop()
    await loop.run_in_executor(executor=None, process, *args: i)
    print(f"end iteration step {i}")
    return i

async def main(): 1 usage
    for j in asyncio.as_completed([f(i) for i in range(10)]):
        result = await j
        print(f"result is {result}")

if __name__ == '__main__':
    asyncio.run(main())
```

See code no. 10



Threading vs. asyncio in Python

- **threading module**
 - No assumptions on libraries, well suited for legacy code
 - Blocking code is safer code
 - Easier to understand
- **asyncio module**
 - Best for massive concurrency (100+ concurrent operations)
 - No overhead for thread creations etc.
 - Best for network requests
 - Good for operations that need to be cancelled (timeouts)
 - Needs async libraries
 - One blocking call can block everything
 - Harder to implement, understand, and maintain